



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	REYES, SOFIA BEATRICE ALEXA R.
Section:	AN42
Professor:	CRISOLA TAN
Date:	10/03/2026
GITHUB LINK:	https://github.com/brryes/Reyes-SofiaBeatriceAlexa-CS0065-AN42.git

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge.

Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware

- A laptop or desktop computer
- Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)

- Any text editor (Notepad++, Sublime, VS Code) if needed
3. File Formats & Tools
- Word processor for documentation:
- MS Word, Google Docs, or LibreOffice
 - Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
(Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:

- Turn on the aircon
- Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [  
    {"problem": {"won't turn on": True, "noise": False},  
     "solution": "Check power cable."}  
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why? **RBR was easier because it uses simple IF-THEN rules.**
2. Which is more flexible in solving unstructured problems? **CBR is more flexible because it uses similar past cases.**
3. Which is better for situations with continuously changing data? **CBR is better for continuously changing data because new cases can be added.**

4. Examples of real-world applications for both. **RBR is used in diagnosis and troubleshooting. CBR is used in customer support, medical cases, and repair systems.**

Your Code - Laboratory Tasks ↴

```
# TA4: Knowledge Representation, RBR, and CBR
# Use Case: Weather Advice System

# =====
# PART 1: KNOWLEDGE REPRESENTATION
# =====

# Knowledge base with five facts
facts = {
    "temperature": 32,
    "is_raining": True,
    "humidity": 85,
    "wind_speed": 20,
    "sky": "cloudy"
}

# Three categories
categories = {
    "environment": ["temperature", "humidity"],
    "weather": ["is_raining", "sky"],
    "safety": ["wind_speed"]
}

print("KNOWLEDGE BASE")
print(facts)

print("\nCATEGORIES")
for category, items in categories.items():
    print(category, ":", items)

# =====
```

```

# PART 2: RULE-BASED REASONING
# =====

def rule_hot(facts):
    if facts["temperature"] >= 30:
        return "Drink plenty of water."

def rule_raining(facts):
    if facts["is_raining"]:
        return "Bring an umbrella."

def rule_humid(facts):
    if facts["humidity"] >= 80:
        return "Stay in a cool place."

def rule_windy(facts):
    if facts["wind_speed"] >= 25:
        return "Be careful because of strong winds."

# Inference engine
def inference_engine(facts, rules):
    actions = []

    for rule in rules:
        result = rule(facts)

        if result is not None:
            actions.append(result)

    return actions

rules = [

```

```

        rule_hot,
        rule_raining,
        rule_humid,
        rule_windy
    ]

rbr_actions = inference_engine(facts, rules)

print("\nRULE-BASED REASONING ACTIONS")

for action in rbr_actions:
    print("-", action)

# =====
# PART 3: CASE-BASED REASONING
# =====

# Three previous cases
case_base = [
    {
        "problem": {
            "hot": True,
            "raining": True,
            "humid": True,
            "windy": False
        },
        "solution": "Bring an umbrella and drink plenty of water."
    },
    {
        "problem": {
            "hot": False,
            "raining": True,
            "humid": False,
            "windy": True
        },
        "solution": "Bring an umbrella and avoid outdoor activities."
    }
]

```

```

    },
    {
        "problem": {
            "hot": False,
            "raining": False,
            "humid": False,
            "windy": False
        },
        "solution": "The weather is comfortable for outdoor activities."
    }
]

# Convert current facts into problem features
new_problem = {
    "hot": facts["temperature"] >= 30,
    "raining": facts["is_raining"],
    "humid": facts["humidity"] >= 80,
    "windy": facts["wind_speed"] >= 25
}

# Calculate similarity
def calculate_similarity(problem1, problem2):
    score = 0

    for feature in problem1:
        if problem1[feature] == problem2[feature]:
            score += 1

    return score

# Retrieve the most similar case
def retrieve_case(new_problem, case_base):
    best_case = None
    highest_score = -1

```



```

    for case in case_base:
        score = calculate_similarity(
            new_problem,
            case["problem"]
        )

        if score > highest_score:
            highest_score = score
            best_case = case

    return best_case, highest_score

# Retrieve and reuse
similar_case, similarity_score = retrieve_case(
    new_problem,
    case_base
)

solution = similar_case["solution"]

print("\nCASE-BASED REASONING")
print("New problem:", new_problem)
print("Most similar case:", similar_case["problem"])
print("Similarity score:", similarity_score)
print("Reused solution:", solution)

# Revise
revision = input(
    "\nEnter a revised solution or press Enter to keep the original: "
)

if revision.strip() != "":
    solution = revision
    print("Solution revised.")
else:

```

```
    print("Original solution retained.")

# Retain
new_case = {
    "problem": new_problem,
    "solution": solution
}

case_base.append(new_case)

print("\nNew case added to the case base.")
print("Total cases:", len(case_base))

print("\nFINAL SOLUTION")
print(solution)
```

```

● PS C:\Users\bea\Documents\elective\ta4> & C:\Users\bea\AppData\Local\Programs\Python\Python313\python.exe c:/Users/bea/Documents/elective/ta4/reasoning.py
KNOWLEDGE BASE
{'temperature': 32, 'is_raining': True, 'humidity': 85, 'wind_speed': 20, 'sky': 'cloudy'}

CATEGORIES
environment : ['temperature', 'humidity']
weather : ['is_raining', 'sky']
safety : ['wind_speed']

RULE-BASED REASONING ACTIONS
- Drink plenty of water.
- Bring an umbrella.
- Stay in a cool place.

CASE-BASED REASONING
New problem: {'hot': True, 'raining': True, 'humid': True, 'windy': False}
Most similar case: {'hot': True, 'raining': True, 'humid': True, 'windy': False}
Similarity score: 4
Reused solution: Bring an umbrella and drink plenty of water.

Enter a revised solution or press Enter to keep the original:
Original solution retained.

New case added to the case base.
Total cases: 4

FINAL SOLUTION
Bring an umbrella and drink plenty of water.
○ PS C:\Users\bea\Documents\elective\ta4> 

```